

LEETCODE 4B:

MIDDLE OF THE LINKED LIST

```
struct ListNode* middleNode(struct ListNode* head) {  
    int length = 0;  
    struct ListNode* temp = head;  
    while (temp != NULL) {  
        length++;  
        temp = temp->next;  
    }  
    int mid = length / 2;  
    temp = head;  
    for (int i = 0; i < mid; i++) {  
        temp = temp->next;  
    }  
    return temp;  
}
```

Output:

Input:
head = [1, 2, 3, 4, 5]

Output:

[3, 4, 5]

LEETCODE 5B:

REMOVE LINKED LIST ELEMENTS

```
struct ListNode* removeElements(struct ListNode* head,  
int val) {  
    struct ListNode* newnode = (struct ListNode*) malloc
```



```

(sizeof (struct ListNode));
newnode → val = 0; // (head)
newnode → next = head;
struct ListNode* current = newnode;
while (current → next != NULL) {
    if (current → next → val == val) {
        struct ListNode* temp = current → next;
        current → next = current → next → next;
        free (temp);
    }
    else {
        current = current → next;
    }
}
struct ListNode* newhead = newnode → next;
free (newnode);
return newhead;
}

```

Input:

head [1, 2, 6, 3, 4, 5, 6]
val = 6.

Output:

[1, 2, 3, 4, 5]

LEETCODE 7B:

LINKED LIST CYCLE

```

bool hasCycle (struct ListNode* head) {
    if (head == NULL || head → next == NULL) {
        return false;
    }

```

```

    struct ListNode* slow = head;

```



```

struct ListNode *fast = head;
while (fast != NULL && fast->next != NULL) {
    slow = slow->next;
    fast = fast->next->next;
}
if (slow == fast) {
    return true;
}
return false;
}

```

Input:
head = [3, 2, 0, -4]
pos = 1

Output:
true

LEETCODE 88: MERGE TWO BINARY TREES

```

struct TreeNode* mergeTrees(struct TreeNode* root1,
struct TreeNode* root2) {
    if (root1 == NULL)
        return root2;
    if (root2 == NULL)
        return root1;
    root1->val = root1->val + root2->val;
    root1->left = mergeTrees(root1->left, root2->left);
    root1->right = mergeTrees(root1->right, root2->right);
    return root1;
}

```

Input:

root 1 = [1, 3, 2, 5]

root 2 = [2, 1, 3, null, 4, null, 7]

Output:

[3, 4, 5, 5, 4, null, 7].

~~Q~~ 23/12/25
Jen